

Računarska vežba 6

PYTHON REČNICI, SKUPOVI, TORKE

1 Rečnici

Rečnici su tipovi podataka slični listama, sa tom razlikom što se rečnici ne indeksiraju brojevima već ključevima koji mogu biti tipa broj, niz ili torka. Vrednosti mogu biti bilo kog tipa i mogu se ponavljati, ali rečnik ne može imati dva ili više istih ključeva.

Rečnik se definiše nabranjem parova **ključ:vrednost** unutar vitičastih zagrada {}, a parovi se međusobno odvajaju zarezom (.). Prazan rečnik označavamo parom praznih vitičastih zagrada. Pristupanje elementima rečnika se vrši kao kod listi, navođenjem ključa između uglastih zagrada.

1.1 Kreiranje rečnika:

Primer:

```
1  # sintaksa za prazan rečnik
2  empty_dict = {}
3  # Rečnik sa podacima
4  dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
```

Primer:

Primer rečnika prikazuje da vrednost može biti bilo kojeg tipa: string, bool, lista, torka ili sam rečnik.

```
1 podaci_programer = {
2     'ime': 'Petar',
3     'prezime': 'Petrović',
4     'godine': 25,
5     'država': 'Srbija',
6     'student': True,
7     'pr_jezici': ['JavaScript', 'C++', 'C', 'SQL', 'Python'],
8     'adresa': {
9         'ulica': 'Travnička 5',
10        'pošt_broj': '15000'
11    }
12 }
```

1.2 Dužina rečnika:

Proverava se broj stavki parova 'key: value' u rečniku.

Primer:

```
1 # sintaksa rečnika
2 dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3 print(len(dct)) # 4
```

Primer:

```
1 podaci_programer = {
2     'ime': 'Petar',
3     'prezime': 'Petrović',
4     'godine': 25,
5     'država': 'Srbija',
6     'student': True,
7     'pr_jezici': ['JavaScript', 'C++', 'C', 'SQL', 'Python'],
8     'adresa': {
9         'ulica': 'Travnička 5',
10        'pošt_broj': '15000'
11    }
12 }
13 print(len(podaci_programer))
```

7

1.3 Pristup članovima rečnika:

Pristup elementima rečnika vrši se referencom ili navođenjem imena ključa. U slučaju pristupa elementu rečnika sa nepostojećim ključem, javlja se greška. Da bi se to izbeglo prvo se proverí postojanje ključa ili se koristi **get** metoda, koja vraća **None**, **NoneType** tip podatka za objekat.

Primer:

```
1 # sintaksa
2 dct = {'key1':'value1', 'key2':'value2', 'key3':'value3', 'key4':'value4'}
3 print(dct['key1']) # value1
4 print(dct['key4']) # value4
```

```
value1
value4
```

Primer:

```
1 programmer = {
2     'ime':'Petar',
3     'prezime':'Petrovic',
4     'godine':25,
5     'drzava':'Srbija',
6     'student':True,
7     'skills':['JavaScript', 'React', 'Node', 'MongoDB', 'Python'],
8     'adresa':{'
9         'ulica':'Travnicka 5',
10        'post_broj':'15000'
11    }
12 }
13 print(programer['ime']) # Petar
14 print(programer['drzava']) # Srbija
15 print(programer['skills']) # ['JavaScript', 'React', 'Node', 'MongoDB', 'Python']
16 print(programer['skills'][0]) # JavaScript
17 print(programer['adresa']['ulica']) # Travnicka 5
18 print(programer['grad']) # Error
```

```
Petar
Srbija
['JavaScript', 'React', 'Node', 'MongoDB', 'Python']
JavaScript
Travnicka 5
Traceback (most recent call last):
  File "c:\Users\ThinkPad\Desktop\Python_vezbe\first.py", line 18, in <module>
    print(programer['grad']) # Error
    ~~~~~^~~~~~
```

```

1  programmer = {
2      'ime': 'Petar',
3      'prezime': 'Petrović',
4      'godine': 25,
5      'drzava': 'Srbija',
6      'student': True,
7      'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python'],
8      'adresa': {
9          'ulica': 'Travnicka 5',
10         'post_broj': '02210'
11     }
12 }
13 print(programer.get('ime')) # Petar
14 print(programer.get('drzava')) # Srbija
15 print(programer.get('skills')) # ['HTML', 'CSS', 'JavaScript', 'React', 'Node', 'MongoDB', 'Python']
16 print(programer.get('grad')) # None

```

```

Petar
Srbija
['JavaScript', 'React', 'Node', 'MongoDB', 'Python']
None

```

1.4 Dodavanje elemenata rečnika:

Uvek je moguće dodati novi ključ-vrednost par u rečnik-

Primer:

```

1  # sintaksa-dodavanje novog kljuca-vrednosti
2
3  dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
4  dct['key5'] = 'value5'

```

Primer:

```

1  programmer = {
2      'ime': 'Petar',
3      'prezime': 'Petrovic',
4      'godine': 25,
5      'drzava': 'Srbija',
6      'student': True,
7      'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python'],
8      'adresa': {
9          'ulica': 'Travnicka 5',
10         'post_broj': '15000'
11     }
12 }
13 programmer['pozicija'] = 'Instruktor'
14 programmer['skills'].append('HTML')
15 print(programer)

```

```

{'ime': 'Petar', 'prezime': 'Petrovic', 'godine': 25, 'drzava': 'Srbija', 'student': True,
'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python', 'HTML'], 'adresa': {'ulica': 'Travnicka 5', 'post_broj': '15000'},
'pozicija': 'Instruktor'}

```

1.5 Promena elemenata rečnika:

Uvek je moguće promeniti elemente rečnika.

Primer:

```
1  programmer = {
2      'ime': 'Petar',
3      'prezime': 'Petrovic',
4      'godine': 25,
5      'drzava': 'Srbija',
6      'student': True,
7      'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python'],
8      'adresa': {
9          'ulica': 'Travnicka 5',
10         'post_broj': '15000'
11     }
12 }
13 programmer['ime'] = 'Marko'
14 programmer['godine'] = 30
15 print(programmer)
```

```
{'ime': 'Marko', 'prezime': 'Petrovic', 'godine': 30, 'drzava': 'Srbija', 'student': True,
'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python'], 'adresa': {'ulica': 'Travnicka 5', 'post_broj': '15000'}}
```

Primer:

```
1  # promena value 1
2  dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3  dct['key1'] = 'value-one'
```

1.6 Provera postojanja ključa u rečniku:

Koristi se operator **in**.

Primer:

```
1  # syntax
2  dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3  print('key2' in dct) # True
4  print('key5' in dct) # False
```

```
True
False
```

1.7 Uklanjanje parova ključ-vrednost iz rečnika:

- *pop(key)*: uklanja element rečnika sa navedenim imenom ključa

- `popitem()`: uklanja poslednji element rečnika
- `del`: uklanja element koji sadrži specificirani ključ

Primer:

```
1 # sintaksa za uklanjanje elemenata rečnika
2 dct = {'key1':'value1', 'key2':'value2', 'key3':'value3', 'key4':'value4'}
3 dct.pop('key1') # uklanja key1
4 dct = {'key1':'value1', 'key2':'value2', 'key3':'value3', 'key4':'value4'}
5 dct.popitem() # uklanja poslednji element
6 del dct['key2'] # uklanja key2
```

Primer:

```
1 programmer = {
2     'ime':'Petar',
3     'prezime':'Petrovic',
4     'godine':25,
5     'drzava':'Srbija',
6     'student':True,
7     'skills':['JavaScript', 'React', 'Node', 'MongoDB', 'Python'],
8     'adresa':{'
9         'ulica':'Travnicka 5',
10        'post_broj':'15000'
11    }}
12 }
13 programmer.pop('ime') # Uklanja ključ ime
14 programmer.popitem() # Uklanja ključ adresa
15 del programmer['student'] # Uklanja student ključ
16 print(programmer)
```

```
{'prezime': 'Petrovic', 'godine': 25, 'drzava': 'Srbija', 'skills': ['JavaScript', 'React', 'Node', 'MongoDB', 'Python']}
```

1.8 Promena rečnika u listu elemenata:

Metoda `items()` menja rečnik u listu torki.

Primer:

```
1 # sintaksa za promenu rečnika u listu torki
2 dct = {'key1':'value1', 'key2':'value2', 'key3':'value3', 'key4':'value4'}
3 print(dct.items()) # dict_items([('key1', 'value1'), ('key2', 'value2'), ('key3', 'value3'), ('key4', 'value4')])
```

1.9 Brisanje elemenata rečnika ili celog rečnika:

Primer:

```
1 # sintaksa za clear
2 dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3 print(dct.clear()) # None
```

Primer:

```
1 # sintaksa brisanje rečnika
2 dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3 del dct
```

1.10 Kopiranje rečnika:

Primer:

```
1 # sintaksa za kopiranje rečnika
2 dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3 dct_copy = dct.copy() # {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
```

1.11 Dobijanje ključeva i vrednosti rečnika kao liste:

Metode *keys()* i *values()* prikazuju sve ključeve i vrednosti rečnika u formi liste.

Primer:

```
1 # sintaksa ključevi kao lista
2 dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3 keys = dct.keys()
4 print(keys) # dict_keys(['key1', 'key2', 'key3', 'key4'])
```

```
dict_keys(['key1', 'key2', 'key3', 'key4'])
```

Primer:

```
1 # sintaksa vrednosti kao lista
2 dct = {'key1': 'value1', 'key2': 'value2', 'key3': 'value3', 'key4': 'value4'}
3 values = dct.values()
4 print(values) # dict_values(['value1', 'value2', 'value3', 'value4'])
```

```
dict_values(['value1', 'value2', 'value3', 'value4'])
```

2 Skupovi

Skup je kolekcija elemenata. Matematička definicija skupa može se primeniti u Python jeziku. Skup je kolekcija neuređenih i neindeksiranih različitih elemenata.

U Python jeziku se skup koristi za čuvanje jedinstvenih elemenata i moguće su operacije unije, preseka, razlike, simetrične razlike, podskupa, nadskupa i disjunktog skupa među skupovima.

Skupovi su poput listi u kojima nije dozvoljeno ponavljanje elemenata (ne dozvoljavaju se duplikati!!!)

Skupovi se definišu vitičastim zagradama {}, dok prazan skup dobijamo konstruktorom *set()*.

2.1 Kreiranje skupova:

Za kreiranje skupova koristi se ugrađena funkcija *set()* ili se elementi skupa inicijalizuju njihovim navođenjem.

Primer:

```
1 # kreiranje skupa
2 st = set()
```

Primer:

```
1 # sintaksa inicijalizacijom elemenata
2 st = {'item1', 'item2', 'item3', 'item4'}

1 # kreiranje skupa
2 items = {'item1', 'item2', 'item3', 'item4'}
```

2.2 Dužina skupova:

Koristi se metoda *len()* da bi se dobila dužina skupa, tj. broj elemenata skupa.

Primer:

```
1 # dužina, broj elemenata skupa
2 st = {'item1', 'item2', 'item3', 'item4'}
3 len(st)
```

2.3 Provera pripadanja elementa skupu:

Za proveru da li se element nalazi u okviru skupa koristi se operator *in*

Primer:

```
1 # da li je element u skupu
2 st = {'item1', 'item2', 'item3', 'item4'}
3 print("Da li skup sadrži item3? ", 'item3' in st) # True
```

Da li skup sadrži item3? True

2.4 Dodavanje novog elementa skupa:

Skup je nepromenljiva struktura podataka (immutable), ne može se menjati vrednost njegovih elemenata, novi elementi se mogu dodavati:

- Korišćenjem metode *add()*
- Korišćenjem metode *update()*

Metodom *update()* može se dodavati više elemenata u skup istovremeno i ova metoda uzima listu argumenata.

Primer:

```
1 # sintaksa za dodavanje elemenata skupa
2 st = {'item1', 'item2', 'item3', 'item4'}
3 st.add('item5')
4 print(st)
```

```
{'item4', 'item3', 'item5', 'item1', 'item2'}
```

Primer:

```
1 # dodavanje elemenata skupa
2 st = {'item1', 'item2', 'item3', 'item4'}
3 st.update(['item5', 'item6', 'item7'])
4 print(st)
```

```
{'item7', 'item1', 'item4', 'item2', 'item3', 'item5', 'item6'}
```

2.5 Uklanjanje elemenata iz skupa:

Uklanjanje bilo kojeg elementa iz skupa vrši se metodom *remove()*. Ako se ne pronade element skupa, metoda *remove()* vraća grešku, pa je potrebno proveriti postojanje elementa u listi. Ako se upotrebi metoda *discard()* ne pojavljuje se grešaka i u slučaju da element ne postoji u skupu a treba da se ukloni.

Primer:

```
1  # uklanjanje elementa skupa
2  st = {'item1', 'item2', 'item3', 'item4'}
3  st.remove('item2')
4  print(st)
```

```
{'item1', 'item4', 'item3'}
```

Primer:

Metoda **pop()** uklanja slučajno izabrani element skupa i vraća uklonjeni element.

```
1  items = {'item1', 'item2', 'item3', 'item4'}
2  items.pop() # uklanja slučajno izabrani element skupa
3  print(items)
```

```
{'item3', 'item1', 'item2'}
```

```
{'item1', 'item3', 'item2'}
```

```
{'item1', 'item4', 'item3'}
```

Primer:

```
1  items = {'item1', 'item2', 'item3', 'item4'}
2  removed_item = items.pop()
3  print(removed_item)
```

```
item4
```

```
item2
```

2.6 Brisanje elemenata iz skupa i brisanje skupa:

Ako želimo isprazniti skup podataka, koristimo metodu *clear* a ako treba obrisati sam skup koristi se *del* operator.

Primer:

```
1  # clear za skup
2  st = {'item1', 'item2', 'item3', 'item4'}
3  st.clear()
4  print(st)
```

```
set()
```

Primer:

```
1  items = {'item1', 'item2', 'item3', 'item4'}
2  del items
3  print(items)
```

```
Traceback (most recent call last):
  File "c:\Users\MakaDaka\Desktop\first.py", line 3, in <module>
    print(items)
    ^^^^^
NameError: name 'items' is not defined. Did you mean: 'iter'?
```

2.7 Konvertovanje liste u skup:

Moguće je konvertovati listu u skup i obrnuto. Konverzija liste u skup uklanja duplikate i ostaju elementi koji se ne ponavljaju.

Primer:

```
1 # Konverzija liste u skup
2 lst = ['item1', 'item2', 'item3', 'item4', 'item1']
3 st = set(lst) # redosled je slučajan, jer je skup neuređen
4 print(st)
```

```
{'item1', 'item3', 'item2', 'item4'}
```

2.8 Spajanje skupova:

Spajanje dva skupa može se izvršiti metodama *union()* ili *update()*

Primer:

```
1 # spajanje skupova
2 st1 = {'item1', 'item2', 'item3', 'item4'}
3 st2 = {'item5', 'item6', 'item7', 'item8'}
4 st3 = st1.union(st2)
5 print(st3)
```

```
{'item8', 'item2', 'item1', 'item3', 'item7', 'item6', 'item5', 'item4'}
```

Primer:

```
1 # spajanje skupova
2 st1 = {'item1', 'item2', 'item3', 'item4'}
3 st2 = {'item5', 'item6', 'item7', 'item8'}
4 st1.update(st2) # elementi skupa st2 dodati skupu st1
5 print(st1)
```

```
{'item3', 'item4', 'item6', 'item8', 'item1', 'item2', 'item7', 'item5'}
```

2.9 Presek skupova:

Primer:

```
1 celi_brojevi = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2 parni_brojevi = {0, 2, 4, 6, 8, 10}
3 celi_brojevi.intersection(parni_brojevi) # {0, 2, 4, 6, 8, 10}
4
5 python = {'p', 'y', 't', 'h', 'o', 'n'}
6 dragon = {'d', 'r', 'a', 'g', 'o', 'n'}
7 python.intersection(dragon) # {'o', 'n'}
```

2.10 Definisanje podskupa i nadskupa:

Skup može biti podskup ili nadskup drugih skupova:

- Podskup, metoda **issubset()**
- Nadskup, metoda **issuperset**

Primer:

```
1 celi_brojevi = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2 parni_brojevi = {0, 2, 4, 6, 8, 10}
3 celi_brojevi.issubset(parni_brojevi) # False, jer je nadskup
4 celi_brojevi.issuperset(parni_brojevi) # True
5
6 python = {'p', 'y', 't', 'h', 'o', 'n'}
7 dragon = {'d', 'r', 'a', 'g', 'o', 'n'}
8 python.issubset(dragon) # False
```

2.11 Razlika, simetrična razlika skupova:

Koristi se metoda **difference()**

Primer:

```
1 # razlika skupova
2 st1 = {'item1', 'item2', 'item3', 'item4'}
3 st2 = {'item2', 'item3'}
4 st2.difference(st1) # set()
5 st1.difference(st2) # {'item1', 'item4'} => st1\st2
```

Primer:

```

1 celi_brojevi = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2 parni_brojevi = {0, 2, 4, 6, 8, 10}
3 celi_brojevi.difference(parni_brojevi) # {1, 3, 5, 7, 9}
4
5 python = {'p', 'y', 't', 'o', 'n'}
6 dragon = {'d', 'r', 'a', 'g', 'o', 'n'}
7 python.difference(dragon) # {'p', 'y', 't'} - neuređeno (karakteristika skupova)
8 dragon.difference(python) # {'d', 'r', 'a', 'g'}
```

Metoda **symmetric_difference()** vraća simetričnu razliku dva skupa, tj. vraća skup koji sadrži sve elemente iz oba skupa, osim zajedničkih elemenata oba skupa, ili matematički: $(A \setminus B) \cup (B \setminus A)$

Primer:

```

1 celi_brojevi = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2 brojevi = {1, 2, 3, 4, 5}
3 celi_brojevi.symmetric_difference(brojevi) # {0, 6, 7, 8, 9, 10}
4
5 python = {'p', 'y', 't', 'h', 'o', 'n'}
6 dragon = {'d', 'r', 'a', 'g', 'o', 'n'}
7 python.symmetric_difference(dragon) # {'r', 't', 'p', 'y', 'g', 'a', 'd', 'h'}
```

2.12 Disjunktni skupovi:

Ako dva skupa nemaju zajedničke elemente nazivamo ih disjunktnim skupovima. Možemo proveriti da li su dva skupa disjunktna koristeći metodu **isdisjoint()**

Primer:

```

1 parni_brojevi = {0, 2, 4, 6, 8}
2 neparni_brojevi = {1, 3, 5, 7, 9}
3 parni_brojevi.isdisjoint(neparni_brojevi) # True, nemaju zajedničke elemente
4
5 python = {'p', 'y', 't', 'h', 'o', 'n'}
6 dragon = {'d', 'r', 'a', 'g', 'o', 'n'}
7 python.isdisjoint(dragon) # False, zajednički elementi {'o', 'n'}
```

3 Torke (tuples)

Torka je kolekcija različitih tipova podataka koja je uređena i nepromenljiva (immutable). Torke se pišu sa običnim zagradama (). Kada se torka kreira, ne možemo menjati vrednosti njenih elemenata, tako da ne postoje metode dodavanja, umetanja i uklanjanja. Za razliku od liste, torka ima malo metoda, od kojih su najvažnije:

- **tuple()**: za kreiranje prazne torke
- **count()**: za brojanje elemenata u torki
- **index()**: za pronalaženje indeksa određenih elemenata u torki
- operator **+**: za spajanje dve ili više torki ili za kreiranje nove torke

3.1 Kreiranje torki:

Kreiranje ili prazne torke ili inicijalizacijom vrednosti elemenata torke.

Primer:

```
1  # prazna torka
2  empty_tuple = ()
3  # upotreba konstruktora
4  empty_tuple = tuple()

1  # torka sa inicijalnim vrednostima elemenata
2  tpl = ('item1', 'item2', 'item3')
```

3.2 Dužina torke:

Metodom *len()* dobija se dužina torke.

Primer:

```
1  # dužina torke
2  tpl = ('item1', 'item2', 'item3')
3  ltp=len(tpl)
4  print(ltp)

3
```

3.3 Pristup elementima torke:

Pozitivnim indeksima, slično kao kod listi. Početni indeks kreće od 0, kao na slici.

(' item1 ', ' item2 ', ' item3 ', ' item4 ')
0 1 2 3

Primer:

```
1 # pristup elementima torke
2 tpl = ('item1', 'item2', 'item3')
3 first_item = tpl[0]
4 second_item = tpl[1]
```

Primer:

```
1 fruits = ('banana', 'orange', 'mango', 'lemon')
2 first_fruit = fruits[0]
3 second_fruit = fruits[1]
4 last_index = len(fruits) - 1
5 last_fruit = fruits[last_index]
```

Negativnim indeksima, slično kao kod listi. Početni indeks kreće od -1, sa desnog kraja torke i nastavlja se u levu stranu, kao na slici.

(' item1 ', ' item2 ', ' item3 ', ' item4 ')
-4 -3 -2 -1

Primer:

```
1 # negativni indeks za torku
2 tpl = ('item1', 'item2', 'item3', 'item4')
3 first_item = tpl[-4]
4 second_item = tpl[-3]
```

Primer:

```
1 fruits = ('banana', 'orange', 'mango', 'lemon')
2 first_fruit = fruits[-4]
3 second_fruit = fruits[-3]
4 last_fruit = fruits[-1]
```

3.4 Razlaganje (slice) torke:

Torka se može razložiti na elemente ili podtorku, upotrebom indeksa i navođenjem početnog i krajnjeg indeksa. Indeksi mogu biti pozitivni ili negativni.

Primer:

```
1  # Slice torke sa pozitivnim indeksima
2  tpl = ('item1', 'item2', 'item3','item4')
3  all_items = tpl[0:4]      # svi elementi
4  all_items = tpl[0:]      # svi elementi
5  middle_two_items = tpl[1:3] # bez elementa na indeksu 3
```

```
1  # Slice torke sa pozitivnim indeksima
2  fruits = ('banana', 'orange', 'mango', 'lemon')
3  all_fruits = fruits[0:4]  # svi elementi
4  all_fruits= fruits[0:]   # svi elementi
5  orange_mango = fruits[1:3] # bez elementa na indeksu 3
6  orange_to_the_rest = fruits[1:]
```

Primer:

```
1  # Slice torke sa negativnim indeksima
2  tpl = ('item1', 'item2', 'item3','item4')
3  all_items = tpl[-4:]     # svi elementi
4  middle_two_items = tpl[-3:-1] # bez elementa sa indeksom 3 index 3 (ili -1)
```

```
1  # Slice torke sa negativnim indeksima
2  fruits = ('banana', 'orange', 'mango', 'lemon')
3  all_fruits = fruits[-4:]  # svi elementi
4  orange_mango = fruits[-3:-1] # bez elementa na indeksu 3
5  orange_to_the_rest = fruits[-3:]
```

3.5 Promena torke u listu:

Torka se može konvertovati u listu i obrnuto. Torka je nepromenljiva ali ako je potrebno promeniti njene elemente, treba je pretvoriti u listu.

Primer:

```
1  # Konverzija torke u listu
2  tpl = ('item1', 'item2', 'item3','item4')
3  lst = list(tpl)
4  print(lst)
```

```
['item1', 'item2', 'item3', 'item4']
```

```
1 # Konverzija torke u listu
2 fruits = ('banana', 'orange', 'mango', 'lemon')
3 fruits = list(fruits)
4 fruits[0] = 'apple'
5 print(fruits)      # ['apple', 'orange', 'mango', 'lemon']
6 fruits = tuple(fruits)
7 print(fruits)      # ('apple', 'orange', 'mango', 'lemon')
```

3.6 Provera elementa torke:

Provera da li se element nalazi u torki vrši se operatorom **in**, koji vraća bool vrednost **True** ili **False**.

Primer:

```
1 # Provera da li je element u torki
2 fruits = ('banana', 'orange', 'mango', 'lemon')
3 print('orange' in fruits) # True
4 print('apple' in fruits) # False
5 fruits[0] = 'apple' # TypeError: 'tuple' object does not support item assignment
```

True
False
Traceback (most recent call last):
 File "c:\Users\MakaDaka\Desktop\first.py", line 5, in <module>
 fruits[0] = 'apple' # TypeError: 'tuple' object does not support item assignment
~~~~~  
TypeError: 'tuple' object does not support item assignment

### 3.7 Spajanje torki:

Koristi se operator **+**.

**Primer:**

```
1 # Spajanje torki
2
3 tp11 = ('item1', 'item2', 'item3')
4 tp12 = ('item4', 'item5', 'item6')
5 tp13 = tp11 + tp12
```

### 3.8 Brisanje torki:

**Primer:**

```
1 # Brisanje torki
2 tp11 = ('item1', 'item2', 'item3')
3 del tp11
```

## Zadaci za vežbu

1) Data je rečnik:

```
inventory = {  
    'gold' : 500,  
    'pouch' : ['flint', 'twine', 'gemstone'],  
    'backpack' : ['xylophone', 'dagger', 'bedroll', 'bread loaf']  
}
```

- a) Dodati ključ pod imenom 'pocket' u rečnik.
- b) Postaviti da vrednost za 'pocket' bude lista koja se sastoji od stringova 'seashell', 'strange berry', i 'lint'
- c) Primeniti `.sort()` na elemente liste u okviru ključa 'backpack'
- d) Zatim ukloniti 'dagger' iz liste pod 'backpack' ključem
- e) Dodati 50 kao vrednost u okviru ključa 'gold'

2) Definirati sledeće rečnike pod nazivom `prices` i `stock` korišćenjem `{}` formata.

- Ključ-vrednost u `prices` rečniku:

```
"banana": 4,  
"apple": 2,  
"orange": 1.5,  
"pear": 3
```

- Ključ-vrednost u `stock` rečniku:

```
"banana": 6,  
"apple": 0,  
"orange": 32,  
"pear": 15
```

Proći kroz svaki ključ u rečniku i svaki ključ prikazati zajedno sa njegovom cenom i informacijama o zalihama. Odgovor treba da bude u sledećem formatu:

```
apple
price: 2
stock: 0
```

Treba izračunati ukupnu zaradu za prodaju celokupnog voća.

- Kreirati promenljivu pod nazivom `total` i postavite je na nulu
- Proći kroz rečnik `prices`. Za svaki ključ u rečniku, treba pomnožiti cenu sa zalihama. Ispisati tu vrednost u konzolu, a zatim je dodati u `total`.
- Na kraju, ispisati ukupnu zaradu `total`

```
1  # Recnik prices
2  prices={}
3  #Dodavanje vrednosti
4  prices["banana"]=4
5  prices["apple"]= 2
6  prices["orange"]= 1.5
7  prices["pear"]= 3
8
9  #Stock recnik
10 stock={}
11 # Dodavanje vrednosti
12 stock["banana"]= 6
13 stock["apple"]= 0
14 stock["orange"] =32
15 stock["pear"]= 15
16
17 #Prikaz cena i zaliha
18
19 for food in prices:
20     print (food)
21     print ("Cena: %s" % prices[food])
22     print ("Zalihe: %s" % stock[food])
23
24     total=0
25 for price in prices:
26     money= prices[price]*stock[price]
27     print (money)
28     total=total +money
29
30 print ("Ukupna zarada je:", total)
```

## 3) Zadatak:

Q What is the expected output of the following code?

```
train_speed = {"FlyingScotsman": 201, "TGV": 320, "Shinkansen": 320}

for train in train_speed.items():
    print(train[0], end="")
```

FlyingScotsmanTGVShinkansen

As the `.items()` method returns a list of tuples where each tuple contains a **(key, value)** pair, the `[0]` index retrieves the first value of each tuple. In effect, only the keys are outputted to the screen.

☒ FTS

☐ 233

☐ The code is erroneous and cannot be run.

## 4) Zadatak:

Q What is **true** about tuples?  
(Select two answers.)

☒ Tuples are immutable, which means that you're not allowed to change their contents.

True: any attempt to change a tuple contents will raise the `TypeError` exception.

☒ The `+` operator can be applied to tuples and means concatenation.

True: for example, `(1,) + (2,)` evaluates to `(1, 2)`.

☐ All tuple elements must be of the same type.

False: tuples of that kind would be very inflexible and of little use.

☐ The `del` keyword can be applied to a single tuple element in order to remove it.

False: immutable beings (including tuples) aren't able to mutate. You can use the `del` keyword to remove a tuple as a whole, but not to change its contents.

## 5) Zadatak:

Data je torka `t = (1, 2, 3, 4, 5)`

Koristeći slice, `reversed()` ili petlju kreirati torku u obrnutom poretku i ispisati je.

...

```
rev = tuple(t[i] for i in range(len(t) - 1, -1, -1))
```

## 6) Zadatak:

Data je lista koja sadrži prazne i neprazne torke. Zadatak je ukloniti sve prazne torke iz liste, kao npr:

Input: `[(1, 2), (), (3, 4), (), (5,)]`

Output: `[(1, 2), (3, 4), (5,)]`

Postoji više načina da se reši zadatak:

1. način

```
a = [(1, 2), (), (3, 4), (), (5,)]  
res = [t for t in a if t]  
print(res)
```

### 2. način

```
a = [(1, 2), (), (3, 4), (), (5,)]  
res = list(filter(None, a))  
print(res)
```

### 3. način

```
a = [(1, 2), (), (3, 4), (), (5,)]  
res = []  
  
for t in a:  
    if t:  
        res.append(t)  
print(res)
```